

Lecture 6:

Function Approximation Classification

Spring 2026

10 ECTS Credits

Department of Information and Communication Technology

Faculty of Engineering and Science

Recap

- Non-parametric plugin classification rule
- Non-parametric classification
 - Histogram Classification
 - k Nearest Neighbor Classification
 - Various distance measures:
 - Euclidean distance
 - Manhattan distance
 - Minkowski distance
 - Mahalanobis distance
 - Hamming distance
 - Kernel Classification



Function Approximation Classification

- In plug-in rules, we estimate:
 - class-conditional density, or
 - posterior probability.
- The decision boundary is dependent on either of these two.
- **Idea:** iteratively adjust the decision boundary by optimizing a criterion that maximizes classification performance.
- Idea behind popular classification rules like
 - support vector machines,
 - neural networks,
 - decision trees,
 - and rank-based classifiers.

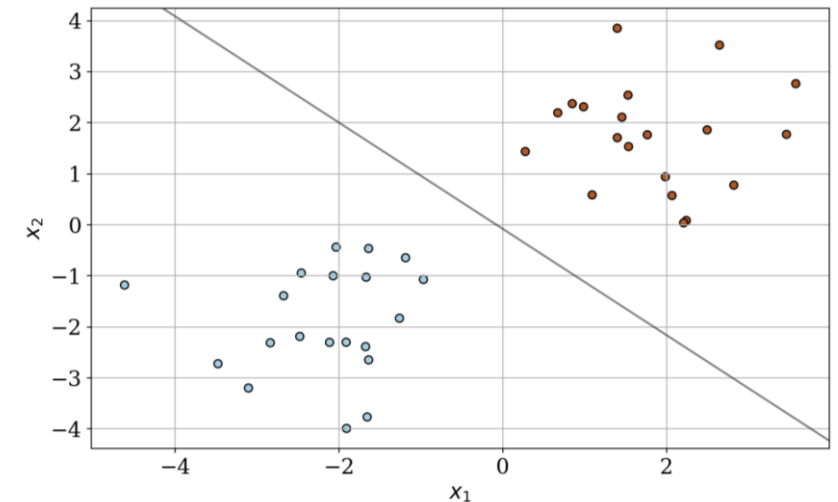
Rosenblatt's Perceptron

- The first function approximation classification rule:

$$\psi(\mathbf{x}) = \begin{cases} 1, & \text{if } a_0 + \sum_{i=1}^d a_i x_i \geq 0 \\ 0, & \text{otherwise} \end{cases}$$

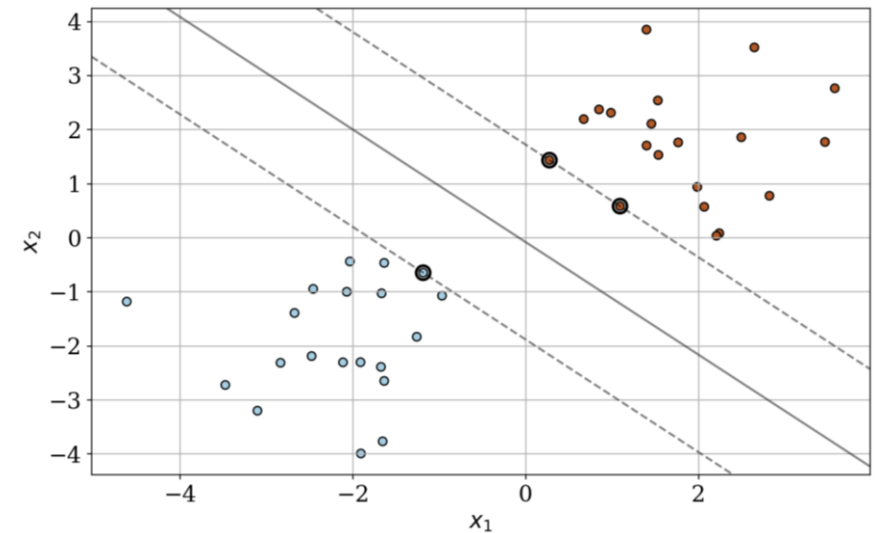
for $\mathbf{x} = (x_1, x_2, \dots, x_d) \in \mathbb{R}^d$.

- Algorithm iteratively adjusts the parameters $a_0, a_1, \dots, a_d$ to maximizes the empirical performance of the classifier.
- Imagine a version of this perceptron that adjusts the decision boundary such that it increases the distance between the separating hyperplane and closest data points.



Linear Support Vector Machines

- A decision boundary such that the distance between the hyperplane and the closest datapoints is maximal.
- This hyperplane is called a **maximum-margin hyperplane**, and the closest points are called **support vectors**.
- The distance between is called **margin**.
- The maximum-margin hyperplane is entirely dependent on the support vectors.
- This classification rule is called linear **SVM**.

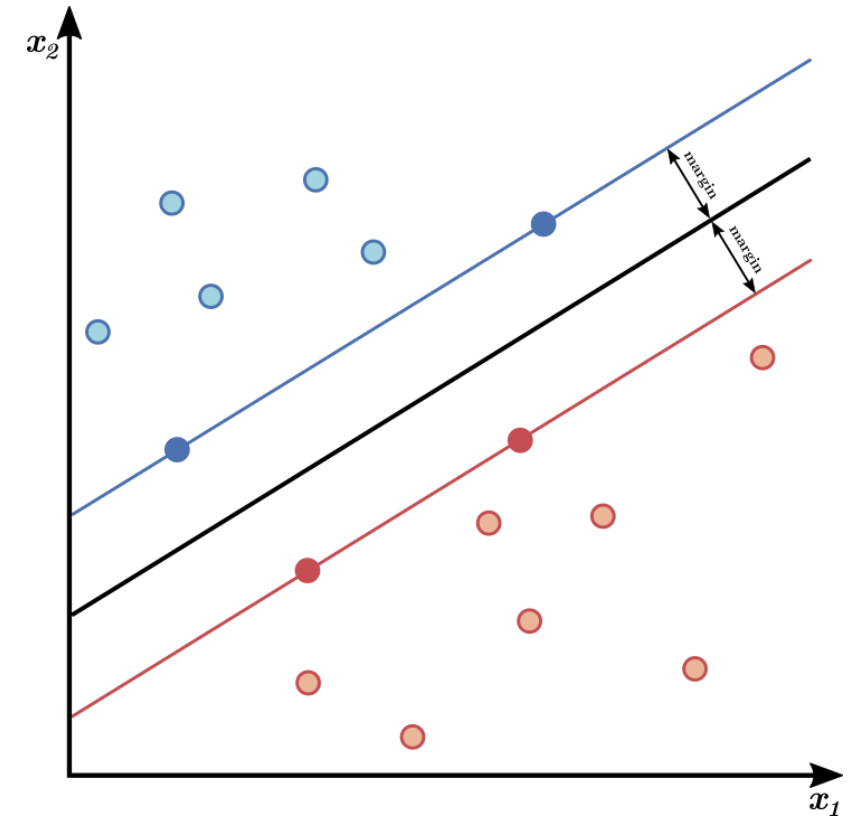


Linear SVMs for Separable Data

- *Assumption:* The data is linearly separable, labels $y_i = \pm 1$.
 - Classification can be done by seeing which side of the decision boundary a point is.
 - $\mathbf{a}^T \mathbf{x}_i + a_0 = 0 \rightarrow$ on the decision boundary,
 - $\mathbf{a}^T \mathbf{x}_i + a_0 > 0 \rightarrow$ classified as +1 (blue),
 - $\mathbf{a}^T \mathbf{x}_i + a_0 < 0 \rightarrow$ classified as -1 (red).
 - Generally, the classification rule is based on:
- $y_i(\mathbf{a}^T \mathbf{x}_i + a_0) > 0$
- To impose a margin, we can change the above constraint to:

$$y_i(\mathbf{a}^T \mathbf{x}_i + a_0) \geq b$$

where $\mathbf{a} = (a_1, a_2, \dots, a_d)$, $i = 1, 2, \dots, n$ and $b > 0$



Linear SVMs for Separable Data

$$y_i(\mathbf{a}^T \mathbf{x}_i + a_0) \geq b$$

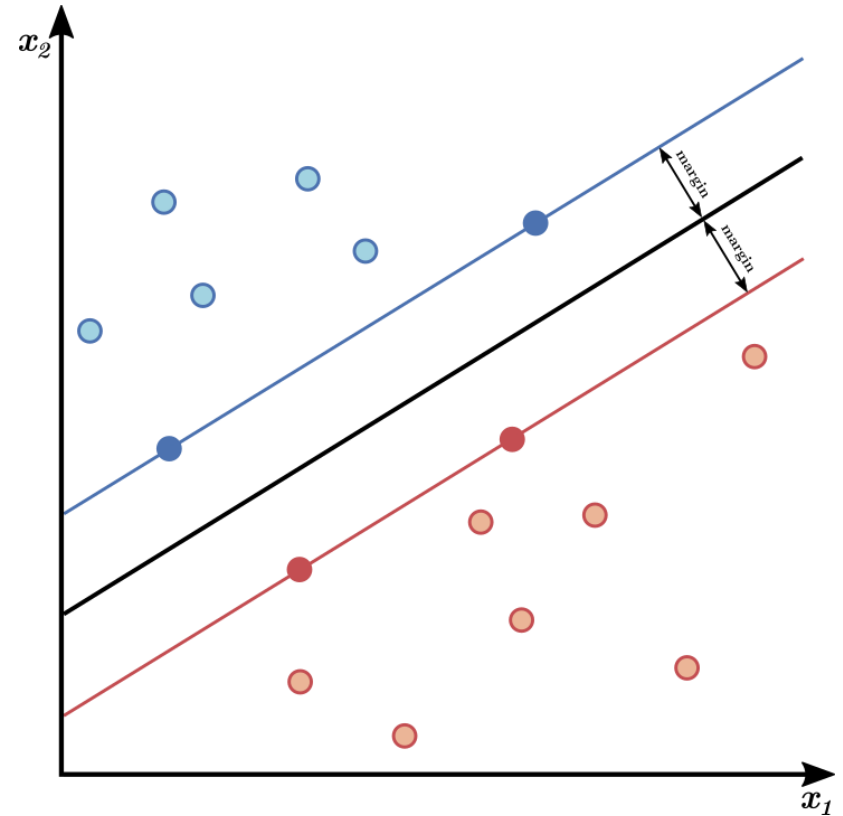
where $\mathbf{a} = (a_1, a_2, \dots, a_d)$, $i = 1, 2, \dots, n$ and $b > 0$

- The distance between a point and the decision hyperplane:

$$Distance = \frac{y_i(\mathbf{a}^T \mathbf{x}_i + a_0)}{\|\mathbf{a}\|}$$

- In other words, each point should be at least distant from the hyperplane by:

$$Distance = \frac{b}{\|\mathbf{a}\|}$$



Linear SVMs for Separable Data

$$Distance = \frac{b}{\|\mathbf{a}\|}$$

where

b is a parameter that controls how far the points must be from the hyperplane and

$\|\mathbf{a}\|$ is the length of vector $\mathbf{a}$, given by $\|\mathbf{a}\| = \sqrt{a_1^2 + \dots + a_d^2}$.

- The parameters $\mathbf{a}$, a_0 , and b can be arbitrarily scaled without changing the constraint, making them not uniquely identifiable. We arbitrarily set $b = 1$.

$$y_i(\mathbf{a}^T \mathbf{x}_i + a_0) \geq 1$$

- The margin becomes $1/\|\mathbf{a}\|$. The points at this exact distance from the decision boundary become margin vectors.

Linear SVMs for Separable Data

- We need to find the maximal-margin hyperplane $\Rightarrow$ maximize the margin $1/\|\mathbf{a}\|$ which is equivalent to minimizing $\|\mathbf{a}\|^2 = \mathbf{a}^T \mathbf{a}$. Thus, the optimization problem becomes :

$$\begin{aligned} \min_{\mathbf{a}, a_0} & \left(\frac{1}{2} \mathbf{a}^T \mathbf{a} \right) \\ \text{s.t. } & y_i(\mathbf{a}^T \mathbf{x}_i + a_0) \geq 1, i = 1, 2, \dots, n \end{aligned} \quad (1)$$

- The idea of SVM is to find the optimal solution to the above problem by adjusting $\mathbf{a}$ and a_0 such that all points in the training set are classified correctly.
- To solve this constrained optimization problem, we use the method of **Lagrange multipliers**:

$$L_p(\mathbf{a}, a_0, \boldsymbol{\lambda}) = \frac{1}{2} \mathbf{a}^T \mathbf{a} - \sum_{i=1}^n \lambda_i (y_i(\mathbf{a}^T \mathbf{x}_i + a_0) - 1) \quad (2)$$

where $\lambda_i \geq 0$ is the Lagrange multiplier for the i^{th} constraint (data point) and $\boldsymbol{\lambda} = (\lambda_1, \dots, \lambda_n)$.

Linear SVMs for Separable Data

- The **primal Lagrangian function** ($L_p(\mathbf{a}, a_0, \lambda)$) codes the constraints into the cost function.
- A solution $(\mathbf{a}^*, a_0^*, \lambda^*)$ to the constrained problem minimizes L_p w.r.t. $(\mathbf{a}, a_0)$ and maximizes w.r.t. λ at the same time.
- Which means we are searching for the **saddle point** in L_p .
- The **Karush-Kuhn-Tucker (KKT) Conditions** are necessary and sufficient for optimality in this convex problem:
 - Stationarity
 - Primal Feasibility
 - Dual Feasibility
 - Complementary Slackness

Linear SVMs for Separable Data

- **Stationarity:** The gradient of $L_p(\mathbf{a}, a_0, \lambda)$ w.r.t. $\mathbf{a}$ and a_0 must vanish:

$$\begin{aligned}\frac{\partial L_p}{\partial \mathbf{a}} = 0 &\Rightarrow \mathbf{a} = \sum_{i=1}^n \lambda_i y_i \mathbf{x}_i \\ \frac{\partial L_p}{\partial a_0} = 0 &\Rightarrow \sum_{i=1}^n \lambda_i y_i = 0\end{aligned}\tag{3}$$

- Substituting these back into $L_p(\mathbf{a}, a_0, \lambda)$, yields the **dual Lagrangian function**:

$$L_D(\lambda) = \sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

- The optimization problem now is:

$$\begin{aligned}\max_{\lambda_i} & L_D(\lambda) \\ \text{s.t. } & \lambda_i \geq 0 \text{ and } \sum_{i=1}^n \lambda_i y_i = 0\end{aligned}$$

- This is a quadratic problem and can be solved numerically.

Linear SVMs for Separable Data

- **Primal Feasibility:**

$$y_i(\mathbf{a}^T \mathbf{x}_i + a_0) - 1 \geq 0$$

- **Dual Feasibility:**

$$\lambda_i \geq 0, \quad \forall i = 1, \dots, n$$

- **Complementary Slackness:**

- If the points do not lie on the margin, the constraint should be inactive ($\lambda_i^* = 0$):

$$y_i((\mathbf{a}^*)^T \mathbf{x}_i + a_0^*) > 1 \text{ (inactive or slack constraint)}$$

- In cases where the points lie on the margin (**support vector** $(\mathbf{x}_i, y_i)$), the constraint should be active ($\lambda_i^* > 0$):

$$y_i((\mathbf{a}^*)^T \mathbf{x}_i + a_0^*) = 1 \text{ (active or tight constraint)}$$

Linear SVMs for Separable Data

- Once we calculate λ^* , the solution vector $\mathbf{a}^*$ is determined by [\(3\)](#):

$$\mathbf{a}^* = \sum_{i=1}^n \lambda_i^* y_i \mathbf{x}_i = \sum_{i \in \mathcal{S}} \lambda_i^* y_i \mathbf{x}_i$$

where $\mathcal{S} = \{i \mid \lambda^* > 0\}$ is the set of all support vector indices.

- Larger $\lambda^* \Rightarrow$ higher influence from the corresponding support vector on the direction vector $\mathbf{a}^*$.
- The intercept a_0^* can be determined from the sum of all support vectors:

$$|\mathcal{S}| a_0^* + (\mathbf{a}^*)^T \sum_{i \in \mathcal{S}} \mathbf{x}_i = \sum_{i \in \mathcal{S}} y_i$$

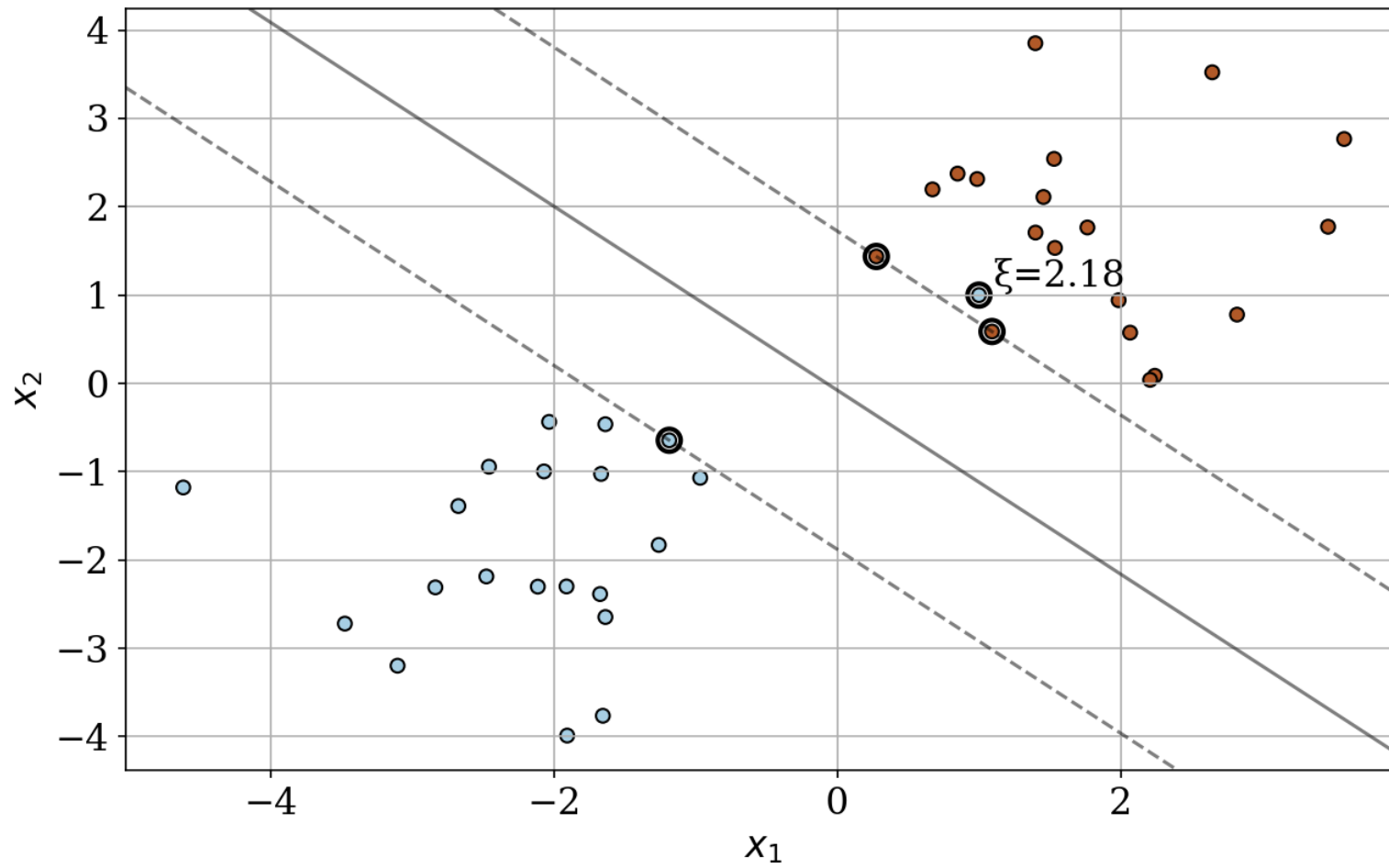
$$a_0^* = -\frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \sum_{j \in \mathcal{S}} \lambda_i^* y_i \mathbf{x}_i^T \mathbf{x}_j + \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} y_i$$

Linear SVMs for Separable Data

- Incorporating all the results and plugging in to the bayes classifier, we have:

$$\psi_n(\mathbf{x}) = \begin{cases} 1, & \text{if } \sum_{i \in \mathcal{S}} \lambda_i^* y_i \mathbf{x}_i^T \mathbf{x} - \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \sum_{j \in \mathcal{S}} \lambda_i^* y_i \mathbf{x}_i^T \mathbf{x}_j + \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} y_i > 0 \\ 0, & \text{otherwise} \end{cases}$$

- Despite the complexity, this is a linear classifier with a hyperplane decision boundary.
- The linear SVM classifier is a function of just the support vectors ($\mathcal{S}$) and other training points don't matter.
- Also known as maximum-margin hyperplane classifier, or hard-margin classifier.



General Linear SVMs

- In practice, it is not possible to guarantee that the training data will be linearly separable.
- So, the linear SVM for linearly separable data can be modified by introducing slack variables $\xi = (\xi_1, \xi_2, \dots, \xi_n)$, one per constraint (totalling $2n$ constraints):

$$y_i(\mathbf{a}^T \mathbf{x}_i + a_0) \geq 1 - \xi_i \text{ and } \xi_i > 0, i = 1, 2, \dots, n$$

- To control slackness, a penalty term is introduced to the optimization problem.

$$\begin{aligned} \min & \left(\frac{1}{2} \mathbf{a}^T \mathbf{a} + C \sum_{i=1}^n \xi_i \right) \\ \text{s.t. } & y_i(\mathbf{a}^T \mathbf{x}_i + a_0) \geq 1 - \xi_i, i = 1, 2, \dots, n \text{ and} \\ & \xi_i \geq 0, i = 1, 2, \dots, n \end{aligned} \tag{4}$$

where, the slack variable, ξ_i measures how much a point violates the margin, and C is a penalty term to keep the slackness under control.

- C helps regulate the sensitivity towards outliers:
 - Small $C \Rightarrow$ softer margin $\Rightarrow$ less overfitting.
 - Large $C \Rightarrow$ harder margin $\Rightarrow$ more overfitting.
 - Too small $C \Rightarrow$ too soft a margin $\Rightarrow$ allowing too much slackness $\Rightarrow$ underfitting.

General Linear SVMs

- Equation [\(4\)](#) is a convex problem in $(\mathbf{a}, a_0, \xi)$.
- Solving similar to the separable case, but there are $2n$ constraints $\Rightarrow 2n$ Lagrange multipliers: $\lambda = (\lambda_1, \lambda_2, \dots, \lambda_n)$ and $\rho = (\rho_1, \rho_2, \dots, \rho_n)$.

- The **primal Lagrangian function** ($L_p(\mathbf{a}, a_0, \xi, \lambda, \rho)$) can be written as:

$$\begin{aligned} L_p(\mathbf{a}, a_0, \xi, \lambda, \rho) &= \frac{1}{2} \mathbf{a}^T \mathbf{a} + C \sum_{i=1}^n \xi_i - \sum_{i=1}^n \lambda_i (y_i (\mathbf{a}^T \mathbf{x}_i + a_0) - 1 + \xi_i) - \sum_{i=1}^n \rho_i \xi_i \\ &= \frac{1}{2} \mathbf{a}^T \mathbf{a} - \sum_{i=1}^n \lambda_i (y_i (\mathbf{a}^T \mathbf{x}_i + a_0) - 1) + \sum_{i=1}^n (C - \lambda_i - \rho_i) \xi_i \end{aligned}$$

- This is equal to the primal Lagrangian function in [\(2\)](#) with an additional term $\sum_{i=1}^n (C - \lambda_i - \rho_i) \xi_i$

General Linear SVMs

- **Stationarity:** The gradient of L_p w.r.t. $\mathbf{a}$ and a_0 must vanish leading to the same equations as in (3).

$$\begin{aligned}\frac{\partial L_p}{\partial \mathbf{a}} = 0 &\Rightarrow \mathbf{a} = \sum_{i=1}^n \lambda_i y_i \mathbf{x}_i, & \forall i = 1, 2, \dots, n \\ \frac{\partial L_p}{\partial a_0} = 0 &\Rightarrow \sum_{i=1}^n \lambda_i y_i = 0, & \forall i = 1, 2, \dots, n\end{aligned}$$

- Setting the derivatives of L_p w.r.t. ξ_i to zero yields:

$$\frac{\partial L_p}{\partial \xi_i} = 0 \Rightarrow C - \lambda_i - \rho_i = 0, \quad \forall i = 1, 2, \dots, n$$

- Substituting these back into L_p , yields the same **dual Lagrangian function**:

$$L_D(\boldsymbol{\lambda}) = \sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

- With the same optimization problem:

$$\begin{aligned}&\max_{\lambda_i} L_D(\boldsymbol{\lambda}) \\ &\lambda_i \geq 0, \quad \sum_{i=1}^n \lambda_i y_i = 0, \quad \lambda_i \leq C, \quad \forall i = 1, \dots, n\end{aligned}$$

- This is a quadratic problem and can be solved numerically to obtain $(\boldsymbol{\lambda}^*, \boldsymbol{\rho}^*)$.

General Linear SVMs

- **Primal feasibility:**

$$y_i(\mathbf{a}^T \mathbf{x}_i + a_0) - 1 + \xi_i \geq 0, \quad \xi_i \geq 0, \quad \forall i$$

- **Dual feasibility:**

$$\lambda_i \geq 0, \quad \rho_i \geq 0, \quad \forall i$$

- **Complementary Slackness:**

$$\lambda_i[y_i(\mathbf{a}^T \mathbf{x}_i + a_0) - 1 + \xi_i] = 0, \quad \rho_i \xi_i = 0, \quad \forall i$$

- In the case of hard-margin classifier, all the support vectors were on the margin, making them a margin vector.
- However, for the general case, in addition to the margin vectors, outliers (through C and ξ) also are considered support vectors as they influence the hyperplane.

General Linear SVMs

- Thus, an **outlier** is a support vector (x_i, y_i) for which:
$$\xi^* > 0 \Rightarrow \rho_i^* = 0 \Rightarrow \lambda_i = C$$
- A **margin vector** is a support vector (x_i, y_i) for which:
$$0 < \lambda_i^* < C$$
- The hard-margin classifier corresponds to the case, where $C = \infty$.
 - No outlier is possible, and
 - all support vectors are margin vectors.
- C helps regulate the sensitivity towards outliers:
 - Small $C \Rightarrow$ more constraints active $\Rightarrow$ more outliers.
 - Large $C \Rightarrow$ less constraints active $\Rightarrow$ less outliers.
 - Too small $C \Rightarrow$ too many constraints active $\Rightarrow$ allowing too much outlier influence.

General Linear SVMs

- As before, the solution vector $\mathbf{a}^*$ is given by:

$$\mathbf{a}^* = \sum_{i=1}^n \lambda_i^* y_i \mathbf{x}_i = \sum_{i \in \mathcal{S}} \lambda_i^* y_i \mathbf{x}_i$$

where $\mathcal{S} = \{i \mid \lambda_i^* > 0\}$ is the set of all support vector indices.

- The intercept a_0^* can be determined from the sum of all margin vectors ($0 < \lambda_i^* < C$):

$$|\mathcal{S}_m| a_0^* + (\mathbf{a}^*)^T \sum_{i \in \mathcal{S}_m} \mathbf{x}_i = \sum_{i \in \mathcal{S}_m} y_i$$

$$a_0^* = -\frac{1}{|\mathcal{S}_m|} \sum_{i \in \mathcal{S}} \sum_{j \in \mathcal{S}_m} \lambda_i^* y_i \mathbf{x}_i^T \mathbf{x}_j + \frac{1}{|\mathcal{S}_m|} \sum_{i \in \mathcal{S}_m} y_i$$

where, $\mathcal{S}_m = \{i \mid 0 < \lambda_i^* < C\} \subseteq \mathcal{S}$ is the margin vector index set.

General Linear SVMs

- The general linear SVM classifier is thus given as:

$$\psi_n(\mathbf{x}) = \begin{cases} 1, & \text{if } \sum_{i \in \mathcal{S}} \lambda_i^* y_i \mathbf{x}_i^T \mathbf{x} - \frac{1}{|\mathcal{S}_m|} \sum_{i \in \mathcal{S}} \sum_{j \in \mathcal{S}_m} \lambda_i^* y_i \mathbf{x}_i^T \mathbf{x}_j + \frac{1}{|\mathcal{S}_m|} \sum_{i \in \mathcal{S}_m} y_i > 0 \\ 0, & \text{otherwise} \end{cases} \quad (5)$$

- A function of support vectors and inner products of the form $\mathbf{x}^T \mathbf{x}$.
- Also known as Soft-margin SVM.

Linear SVM Variants

- Hard-Margin SVM (Maximum-Margin Classifier)
 - Also known as vanilla linear SVM
 - Requires perfect linear separation
 - No tolerance for misclassification
 - Uses support vectors that lie exactly on the margin.
- Soft-Margin SVM (General Linear SVM)
 - Introduces slack variables to handle non-separable data
 - Allows some points to violate the margin
 - Uses penalty parameters to control margin violations
 - Also known as large margin classifier

Non-linear SVM

- Apply the general form of linear SVM on a transformed space $\mathbb{R}^p, p \gg d$.
- Non-linear data can be made linearly separable in higher-dimensional space.
- Let $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^p$ denote the transformation. The derivation in the general case of linear SVM is unchanged with $\phi(x)$ replacing x .
- The SVM classifier in the original feature space $\mathbb{R}^d$ is given by:

$$\psi_n(x) = \begin{cases} 1, & \text{if } \sum_{i \in \mathcal{S}} \lambda_i^* y_i \phi(x_i)^T \phi(x) - \frac{1}{|\mathcal{S}_m|} \sum_{i \in \mathcal{S}} \sum_{j \in \mathcal{S}_m} \lambda_i^* y_i \phi(x_i)^T \phi(x_j) + \frac{1}{|\mathcal{S}_m|} \sum_{i \in \mathcal{S}_m} y_i > 0 \\ 0, & \text{otherwise} \end{cases} \quad (6)$$

- In the original feature space, generally this produces a non-linear decision boundary.

Non-linear SVM

- The calculation of the transformation explicitly and later calculation of $\phi(\mathbf{x}_i)^T \phi(\mathbf{x})$ is computationally expensive and impractical.
- Thus, we use **Kernel Trick**.
- Introducing a kernel function:
$$k(\mathbf{x}, \mathbf{x}') = \phi^T(\mathbf{x})\phi(\mathbf{x}'), \quad \mathbf{x}, \mathbf{x}' \in \mathbb{R}^d$$
- The kernel trick is to avoid calculating $\phi(\mathbf{x})$ completely by using $k(\mathbf{x}, \mathbf{x}')$.
- Possible because the linear SVM classifier in [\(5\)](#) is only a function of the form $\mathbf{x}^T \mathbf{x}$ which makes the non-linear SVM classifier a function of $k(\mathbf{x}, \mathbf{x}') = \phi^T(\mathbf{x})\phi(\mathbf{x}')$.

Non-linear SVM

- We need to maximize the dual function:

$$\max_{\lambda} \left[L_D(\lambda) = \sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j y_i y_j k(\mathbf{x}_i, \mathbf{x}_j) \right]$$

s.t. $0 \leq \lambda_i \leq C$ and $\sum_{i=1}^n \lambda_i y_i = 0$

- Once we find out λ^* , the non-linear SVM can be defined as:

$$\psi_n(\mathbf{x}) = \begin{cases} 1, & \text{if } \sum_{i \in \mathcal{S}} \lambda_i^* y_i k(\mathbf{x}, \mathbf{x}') - \frac{1}{|\mathcal{S}_m|} \sum_{i \in \mathcal{S}} \sum_{j \in \mathcal{S}_m} \lambda_i^* y_i k(\mathbf{x}, \mathbf{x}') + \frac{1}{|\mathcal{S}_m|} \sum_{i \in \mathcal{S}_m} y_i > 0 \\ 0, & \text{otherwise} \end{cases}$$

- This is a kernel based classification but only dependent on the support vectors.
- The kernel is also weighted by the corresponding support vector weighting parameter λ_i^*

Non-linear SVM

- It is not necessary to construct a kernel specifying $\phi(\mathbf{x})$.
- The function $k(\mathbf{x}, \mathbf{x}')$ can be specified directly if it is expressible as an inner product $\phi^T(\mathbf{x})\phi(\mathbf{x}')$ in some space.
- This happens iff.:
 - k is symmetric: $k(\mathbf{x}, \mathbf{x}') = k(\mathbf{x}', \mathbf{x}), \forall \mathbf{x}, \mathbf{x}' \in \mathbb{R}^d$
 - k is positive semidefinite:

$$\int k(\mathbf{x}, \mathbf{x}')g(\mathbf{x})g(\mathbf{x}')d\mathbf{x}d\mathbf{x}' \geq 0,$$

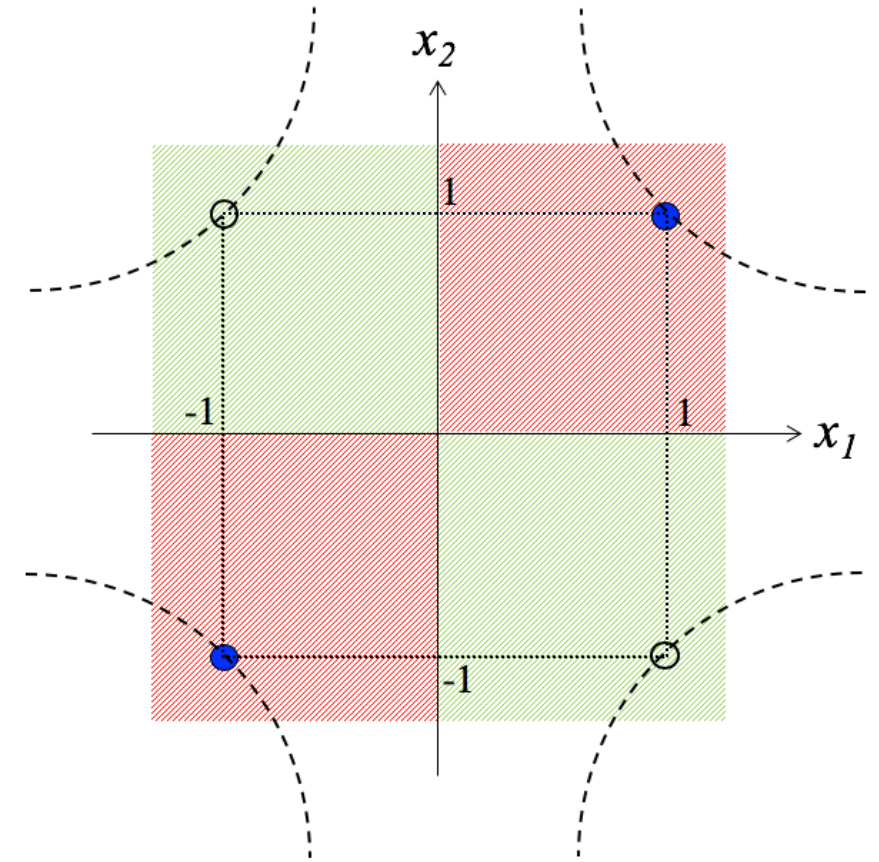
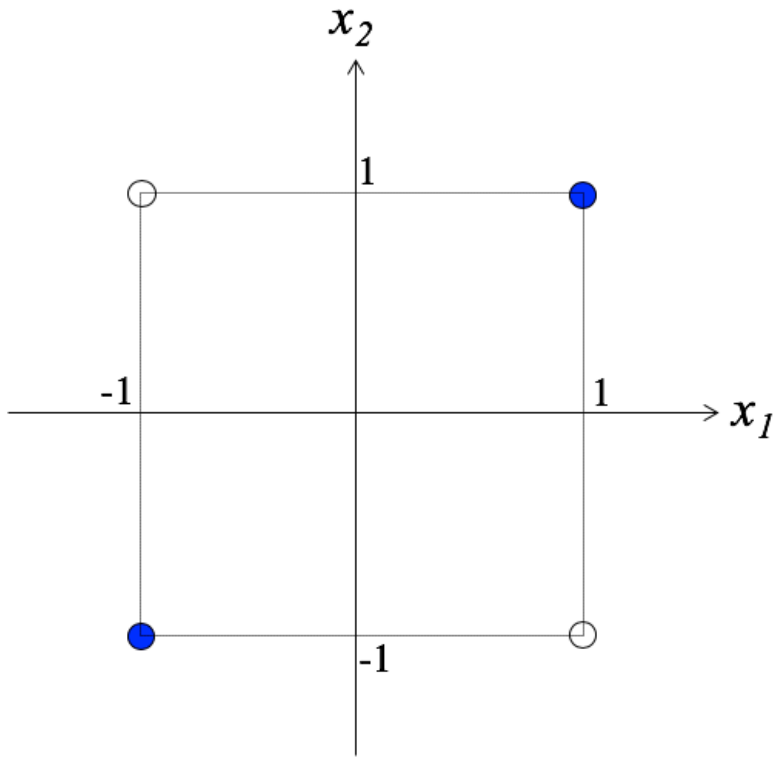
for any square-integrable function $g: \mathbb{R}^d \rightarrow \mathbb{R}$ i.e. $(\int g^2(\mathbf{x})d\mathbf{x} < \infty)$

- This is known as **Mercer's Theorem** and the conditions above are called **Mercer's conditions**.

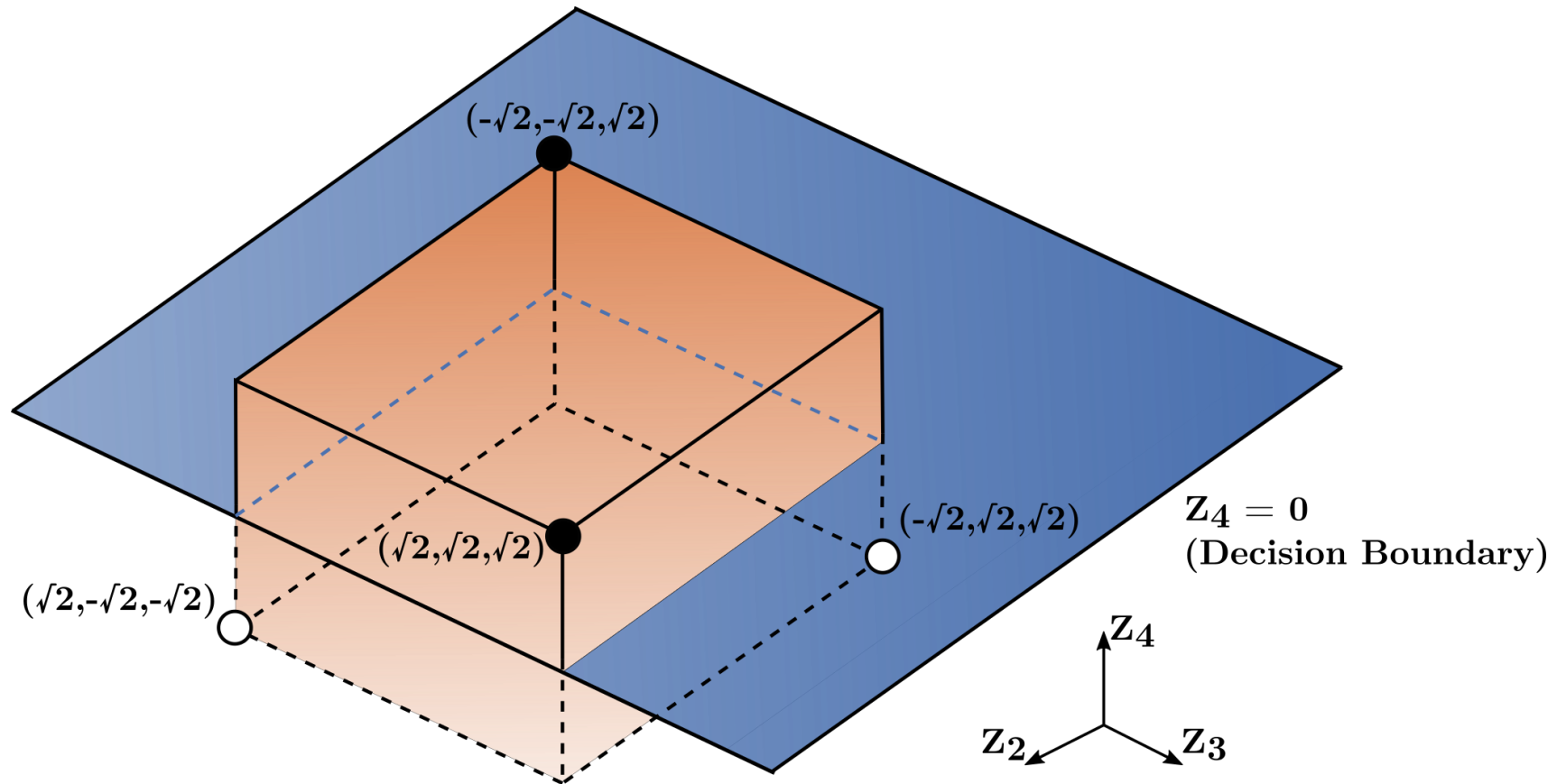
Non-linear SVM

- Some kernels used in practice are:
 - Polynomial kernel: $k(\mathbf{x}, \mathbf{x}') = (1 + \mathbf{x}^T \mathbf{x}')^p, \forall p = 1, 2, \dots$
 - Gaussian kernel: $k(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2), \forall \gamma > 0.$
 - Sigmoid kernel: $k(\mathbf{x}, \mathbf{x}') = \tanh(\gamma \mathbf{x}^T \mathbf{x}' - \delta).$
- All the above conditions satisfy Mercer's condition.
- The gaussian kernel is called the **radial basis function (RBF) kernel** in the context of SVMs.
- In the case of the Gaussian kernel, the mapping ϕ is into an infinite-dimensional space, so it cannot be computed exactly (but we do not need to).

Non-linear SVM : Example

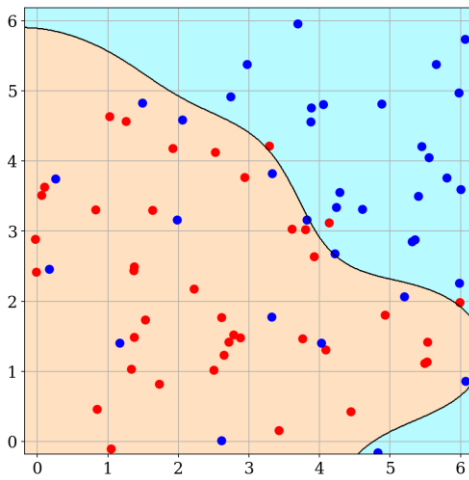


Non-linear SVM : Example

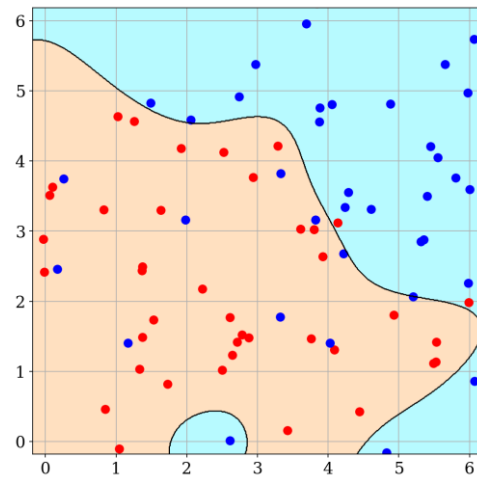


Non-linear SVM: Example

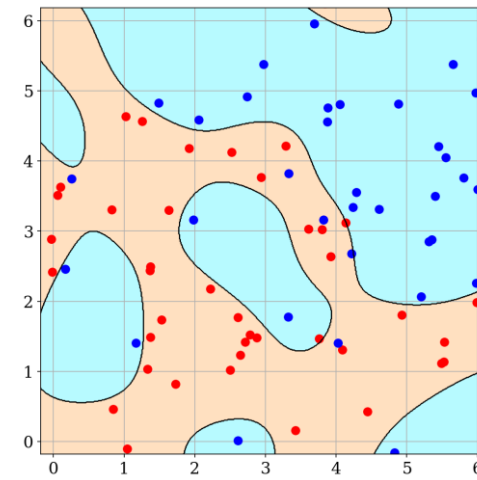
- The Gaussian-RBF kernel with $\gamma = 0.5$ and varying C is used below.
- With larger values of C , we penalize presence of outliers, hence overfitting.
- With $C = 1$, the decision boundary is close to the optimal one.
- The choice of C is a model selection problem.



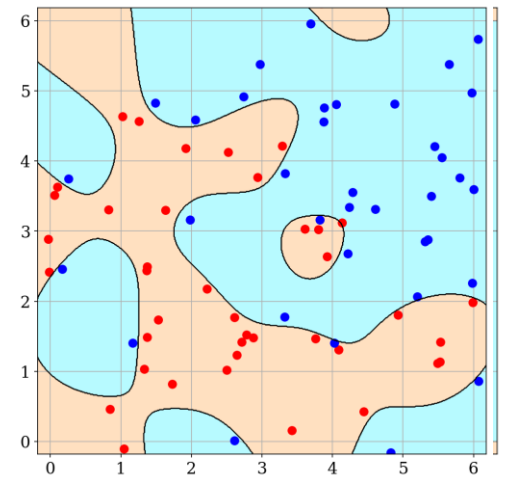
$C = 1$



$C = 10$



$C = 100$

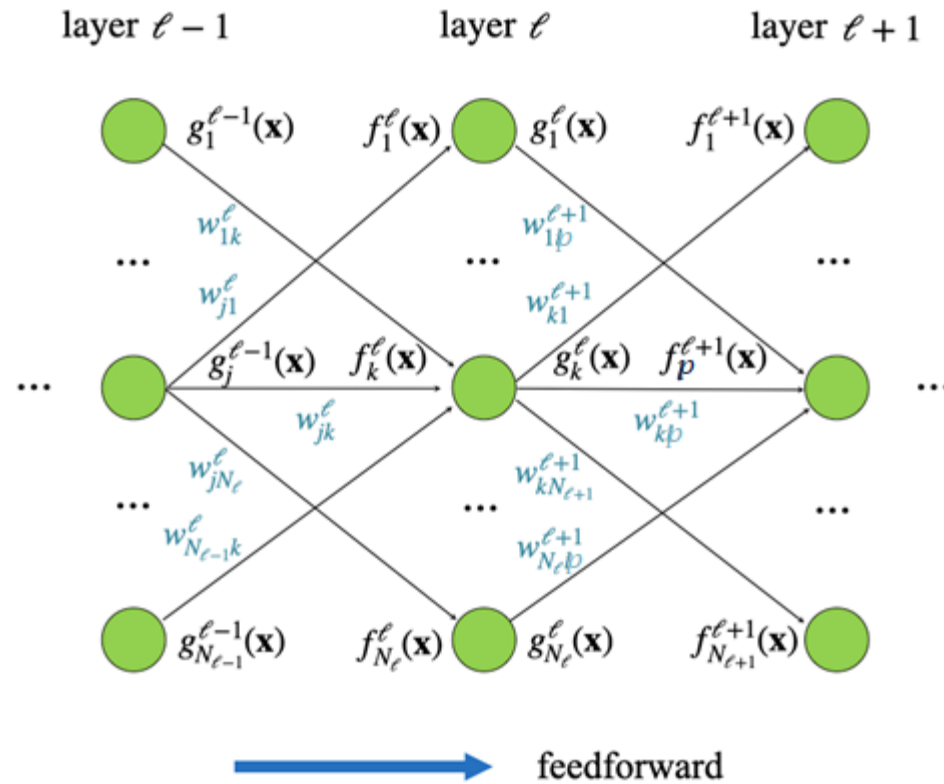


$C = 1000$

Neural Networks

- Combines linear function and univariate **nonlinearities** to produce complex discriminants.
- Consists of units called **neurons**, which are organized in **layers**.
- The **architecture** specifies how the neurons are connected.
- The typical architecture consists of an input layer, hidden layer(s), and an output layer.
- A **deep neural network** is, generally, an NN with more than one **hidden layer**.
- The basic architecture is **fully-connected**: all neurons of a layer l are connected to all the neurons in the layer $l + 1$.

A Fully Connected Neural Network



Neural Networks

- An NN with d inputs, L layers, m outputs, with nonlinearity $\sigma: \mathbb{R} \rightarrow \mathbb{R}$, and N_l neurons per layer can be represented as:

$$f_k^l(\mathbf{x}) = w_{0k}^l + \sum_{j=1}^{N_{l-1}} w_{jk}^l g_j^{l-1}(\mathbf{x}), \quad l = 1, 2, \dots, L, \quad k = 1, \dots, N_l$$
$$g_k^l(\mathbf{x}) = \begin{cases} \sigma(f_k^l(\mathbf{x})), & l = 1, 2, \dots, L-1; \quad k = 1, \dots, N_l \\ f_k^l(\mathbf{x}), & l = L, \quad k = 1, \dots, N_L \\ x_k, & l = 0, \quad k = 1, \dots, d \end{cases} \quad (6)$$

where $\mathbf{x} \in \mathbb{R}^d$ is the input, $N_0 = d$, $N_L = m$. The **weight vector** is

$$\mathbf{w} = [w_{jk}^l, \quad l = 1, 2, \dots, L, \quad j = 0, \dots, N_{l-1}, \quad k = 1, \dots, N_l]$$

- The subset of weights with $j = 0$ are called **bias**. Together, they are known as **parameters**.

Neural Networks

- Total bias parameters $\sum_{l=1}^L N_l$
- Total non-bias parameters $\sum_{l=1}^L N_{l-1} N_l$
- The number of weights in a fully-connected NN
 - grows linearly with network **depth**, L .
 - grows quadratically with network **width**, N_l .
- The functions $f_k^l(x)$ are the **pre-activations** of layer l .
- Layers $1, \dots, L - 1$ are the **hidden layers** and layer L is the **output layer**.
- The final outputs are the pre-activations of the final layer: $f_j^L(x)$.
- The nonlinearities used $\sigma: \mathbb{R} \rightarrow \mathbb{R}$, can be two types:
 - Saturating non-linearities,
 - Non-saturating non-linearities.

Neural Networks (Saturating Non-Linearities)

- Step non-linearity:

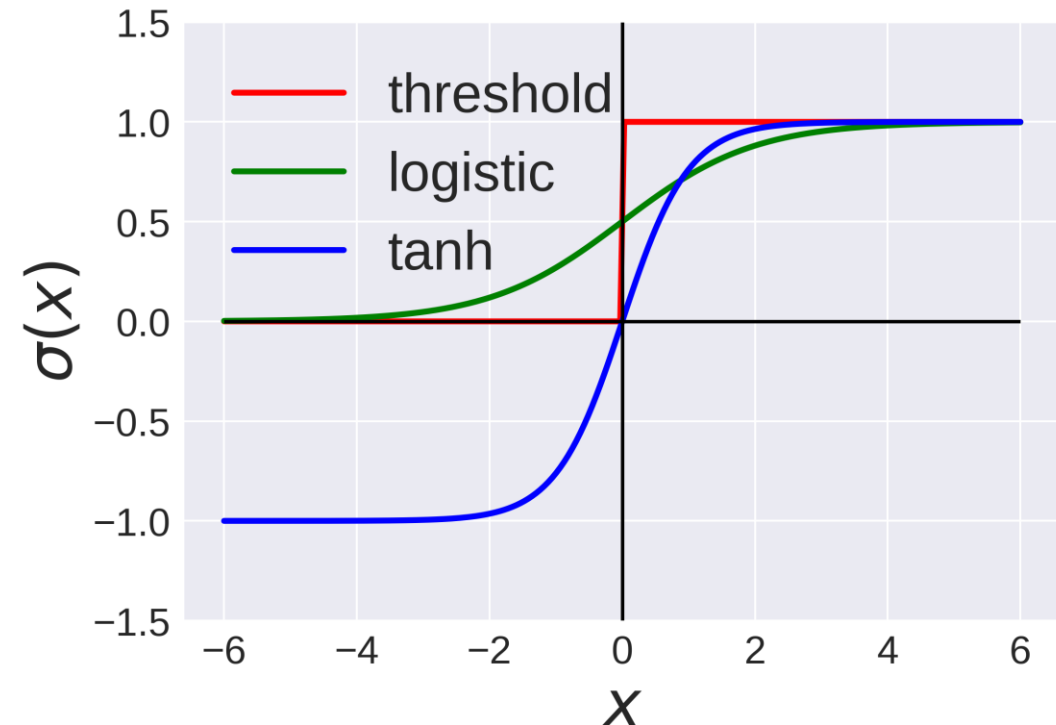
$$\sigma(x) = \begin{cases} 1, & \text{if } x > 0, \\ 0, & \text{if } x \leq 0. \end{cases}$$

- Logistic non-linearity:

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

- Hyperbolic tangent (tanh) non-linearity:

$$\sigma(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}.$$



Neural Networks (Non-saturating Non-linearities)

- Rectifier linear unit (ReLU) non-linearity:

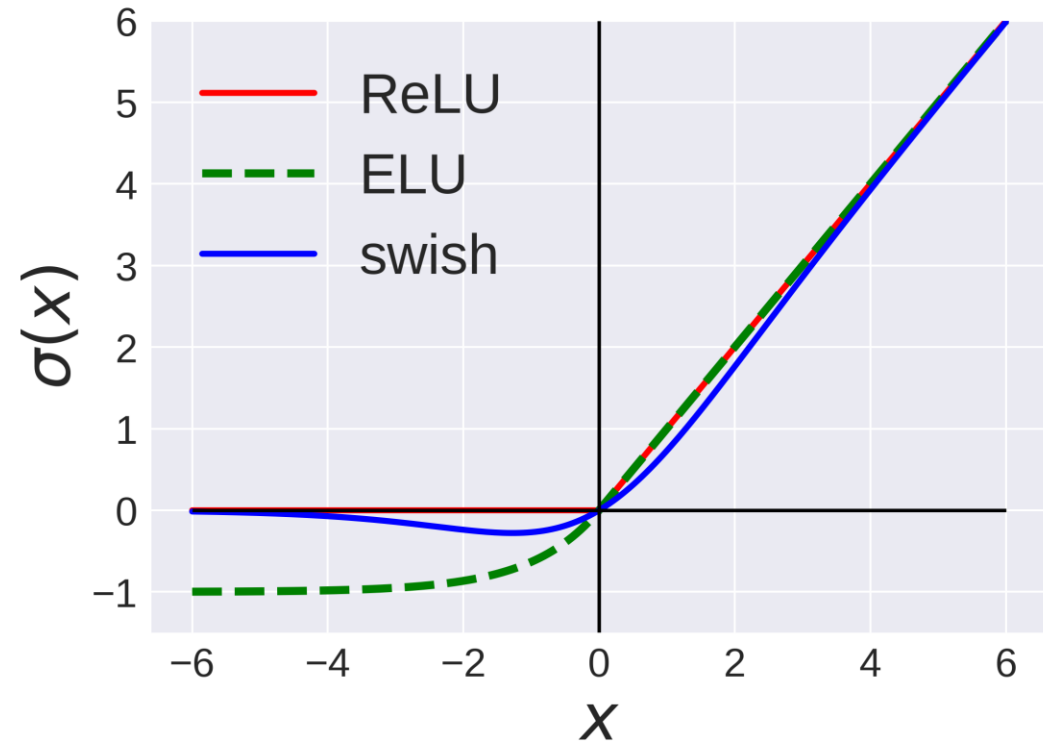
$$\sigma(x) = \max(x, 0) = \begin{cases} x, & \text{if } x > 0, \\ 0, & \text{if } x \leq 0. \end{cases}$$

- Exponential linear unit (ELU) non-linearity:

$$\sigma(x) = \begin{cases} x, & \text{if } x > 0, \\ \alpha(e^x - 1), & \text{if } x \leq 0 (\alpha > 0). \end{cases}$$

- Swish non-linearity:

$$\sigma(x) = \frac{x}{1 + e^{-x}}.$$



Neural Network Classifiers

- Approximating a function that maps d real-valued inputs to m real-valued outputs.
- When $c > 2$, the number of output $m = c$, each of which is passed through a **softmax layer**:

$$\zeta_k(\mathbf{x}) = \frac{e^{f_k(\mathbf{x})}}{\sum_{k=0}^{c-1} e^{f_k(\mathbf{x})}}, \quad k = 0, \dots, c-1,$$

- $\zeta_k(\mathbf{x}) \in [0,1]$, for $k = 0, \dots, c-1$, and $\sum_{k=0}^{c-1} \zeta_k(\mathbf{x}) = 1$.
- A classifier is obtained by selecting the class corresponding to the largest probability.
- When $c=2$, logistic non-linearity may be used:

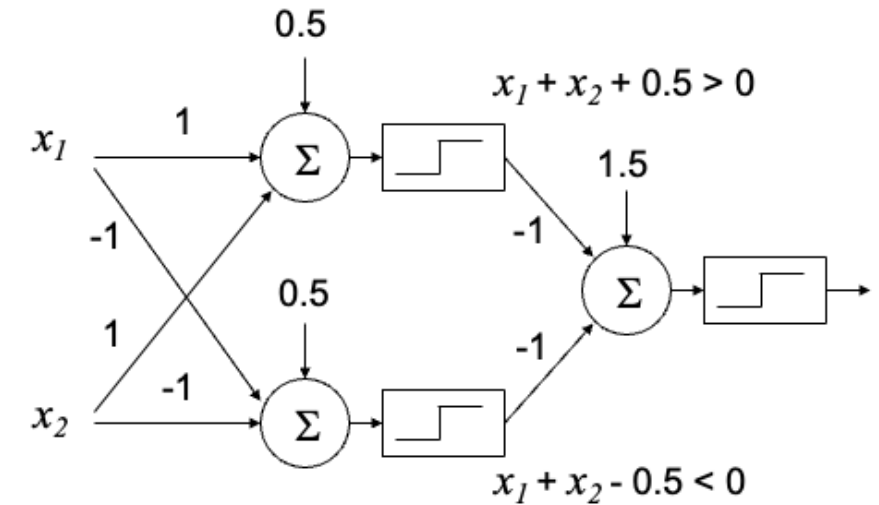
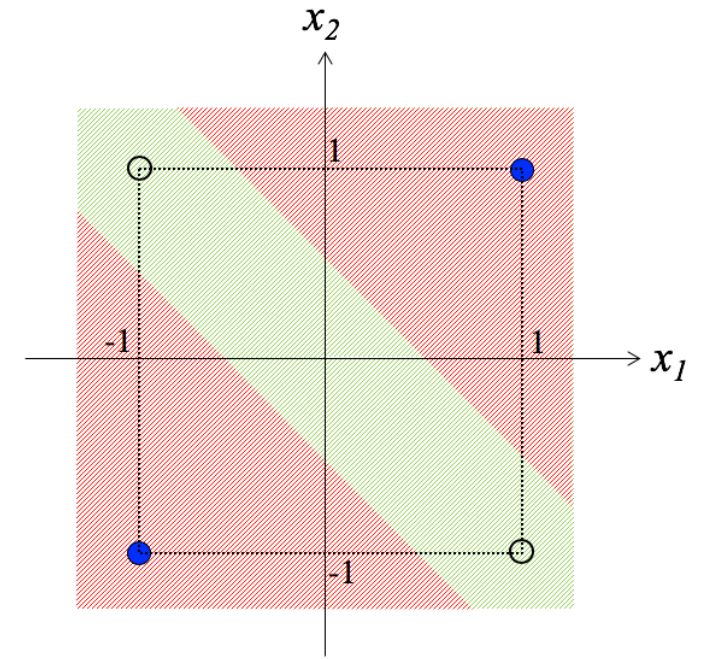
$$\zeta(\mathbf{x}) = \frac{1}{1 + e^{-f(\mathbf{x})}}.$$

Neural Network Classifiers

- Suppose logistic non-linearity is not used.
- The output $f(x)$ is trained as a classic discriminant and threshold set at an appropriate value to produce 0 and 1.
- The output non-linearity is a step function, and the output layer is a perceptron.
- This is similar to nonlinear SVM classifier:
 - Hidden layers maps original feature into a different space,
 - Output layer acts on the transformed feature by using a linear discriminant.
 - The decision in original input space is non-linear.

Neural Network Classifiers

- First hidden neuron:
 - $h_1 = \text{step}(x_1 + x_2 + 0.5)$
 - A decision boundary at $x_1 + x_2 + 0.5 = 0$
 - Rearranging, $x_2 = -x_1 - 0.5$
 - A diagonal line with a slope (-1) s.t.
 - $h_1 = 1$ above the line,
 - $h_1 = 0$ below the line.
- Second hidden neuron:
 - $h_2 = \text{step}(-x_1 - x_2 + 0.5)$
 - A decision boundary at $-x_1 - x_2 + 0.5 = 0$
 - Rearranging, $x_2 = -x_1 + 0.5$
 - A diagonal line with a slope (-1) s.t.
 - $h_2 = 0$ above the line,
 - $h_2 = 1$ below the line.
- Output neuron:
 - $y = \text{step}(-h_1 - h_2 + 1.5)$
 - Combines the above two decision boundaries to create a decision region s.t.
 - $y = 0$, when h_1 and h_2 are both +1
 - $y = 1$. otherwise



Loss Functions

- Neural network training is an attempt to find the weights that minimize a training loss function $\mathcal{L}(\mathbf{w})$.
- Iteratively minimizes the loss, which quantifies the mislabelling of training data.
- Using softmax layer, the outputs $\zeta_k(\mathbf{x})$ will always be probabilities such that $\sum_{k=0}^{c-1} \zeta_k(\mathbf{x}) = 1, \forall k = 0, 1, 2, \dots, c - 1$.
- If a training point is defined as $(\mathbf{x}, y)$, we need to penalize a lower probability corresponding to the real class y , $\zeta_y(\mathbf{x})$ and higher probability for the other classes.
- Given training data, $S_n = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_n, y_n)\}$, the **log loss function** can be defined as:

$$\mathcal{L}(\mathbf{w}) = - \sum_{i=1}^n \log \zeta_{y_i}(\mathbf{x}_i)$$

Loss Functions

- The log loss function is also called a **cross-entropy loss**.
- It measures the cross-entropy (a measure of difference between two probability distribution) quantifying the loss of information when one distribution approximates another.
- We are approximating the true distribution of classes $p(\mathbf{x})$ using $\zeta(\mathbf{x})$ where:
 - $p_j(\mathbf{x}) = 1$, if $j = y$ is the true class.
 - $p_j(\mathbf{x}) = 0$, for all the other classes.
- The log-loss is thus the sum of cross-entropies between the discrete distributions $p(\mathbf{x})$ and $\zeta(\mathbf{x})$:

$$\mathcal{L}(\mathbf{w}) = - \sum_{i=1}^n \sum_{k=0}^{c-1} p_k(\mathbf{x}_i) \log \zeta_k(\mathbf{x}_i; \mathbf{w}).$$

- For binary classification,

$$\mathcal{L}(\mathbf{w}) = - \sum_{i=1}^n y_i \log \zeta(\mathbf{x}_i; \mathbf{w}) + (1 - y_i) \log(1 - \zeta(\mathbf{x}_i; \mathbf{w}))$$

Backpropagation Algorithm

- Given a loss function $\mathcal{L}(\mathbf{w})$, we want to minimize:

$$\min_{\mathbf{w}} \mathcal{L}(\mathbf{w}), \quad \forall \mathbf{w} \in \mathbb{R}^p$$

- The most common approach is via **gradient descent**, where we start with a random initialized weight vector $\mathbf{w}_0$ and recursively calculate:

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \eta_t \nabla \mathcal{L}(\mathbf{w}^t) \quad (7)$$

where $t = 0, 1, \dots; \eta_t > 0$ is the **learning rate**, and the gradient of loss function:

$$\nabla \mathcal{L}(\mathbf{w}) = \left[\frac{\partial \mathcal{L}(\mathbf{w})}{\partial w_{jk}^l}, \quad l = 1, 2, \dots, L, \quad j = 0, 1, \dots, N_{l-1}, \quad k = 1, \dots, N_l \right] \quad (8)$$

- The partial derivatives are calculated using **backpropagation algorithm**.

Backpropagation Algorithm

- The function is of the form:

$$\mathcal{L}(\mathbf{w}) = \sum_{i=1}^n \mathcal{L}_i(\mathbf{w})$$

- Hence, the partial derivatives $\frac{\partial \mathcal{L}}{\partial w_{jk}^l}$ can be written as:

$$\frac{\partial \mathcal{L}}{\partial w_{jk}^l} = \sum_{i=1}^n \frac{\partial \mathcal{L}_i}{\partial w_{jk}^l}$$

- Using chain rule,

$$\frac{\partial \mathcal{L}_i}{\partial w_{jk}^l} = \frac{\partial \mathcal{L}_i}{\partial f_k^l} \frac{\partial f_k^l}{\partial w_{jk}^l}$$

- Representing $\bar{f}_k^l = \frac{\partial \mathcal{L}_i}{\partial f_k^l}$ and using [\(6\)](#):

$$\frac{\partial \mathcal{L}_i}{\partial w_{jk}^l} = \bar{f}_k^l g_j^{l-1}(x_i)$$

Backpropagation Algorithm

- The activation **adjoint** $\bar{f}_k^l$ can be recursively calculated using chain rule:

$$\bar{f}_k^l = \frac{\partial \mathcal{L}_i}{\partial f_k^l} = \sum_{p=1}^{N_{l+1}} \frac{\partial \mathcal{L}_i}{\partial f_p^{l+1}} \frac{\partial f_p^{l+1}}{\partial f_k^l} = \sum_{p=1}^{N_{l+1}} \bar{f}_p^{l+1} \frac{\partial f_p^{l+1}}{\partial f_k^l}$$

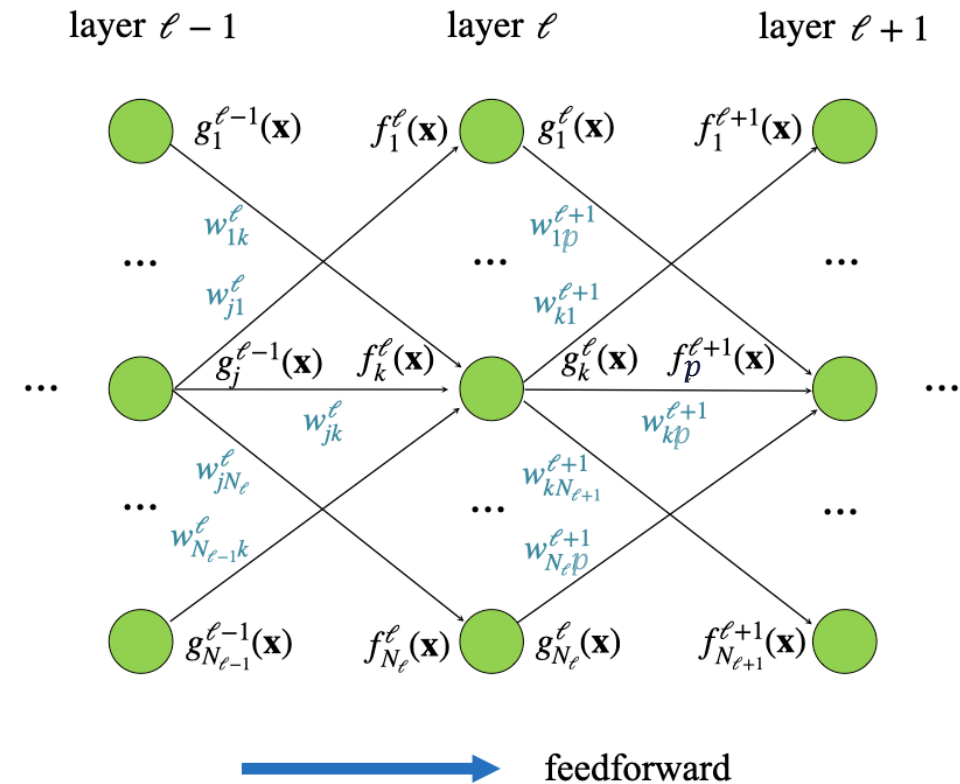
where,

$$\frac{\partial f_p^{l+1}}{\partial f_k^l} = \frac{\partial f_p^{l+1}}{\partial g_k^l} \frac{\partial g_k^l}{\partial f_k^l} = w_{kp}^{l+1} \sigma' (f_k^l(x_i))$$

- The resultant backpropagation equation is:

$$\bar{f}_k^l = \sigma' (f_k^l(x_i)) \sum_{p=1}^{N_{l+1}} w_{kp}^{l+1} \bar{f}_p^{l+1} \quad (9)$$

- That is, the adjoints of one hidden layer is backpropagated from adjoints of the next layer.



Backpropagation Procedure

Given a training input $\mathbf{x}_i$ with label y_i

- The current NN with current weights $\mathbf{w}$ runs forward and calculates:
 - Pre-activations, $f_k^l(\mathbf{x}_i)$, and
 - Activations $g_k^l(\mathbf{x}_i)$.
- The adjoints $\bar{f}^L$ are calculated for the last layer L .

$$\bar{f}^L = \frac{\partial \mathcal{L}_i}{\partial f^L} = \frac{\partial}{\partial f^L} \left(\frac{1}{2} (y_i - f^L(\mathbf{x}_i))^2 \right) = -(y_i - f^L(\mathbf{x}_i))$$

Even though the calculation is this, we use $y_i - f^L(\mathbf{x}_i)$ and the negative sign is taken care of in [\(7\)](#).

Backpropagation Procedure

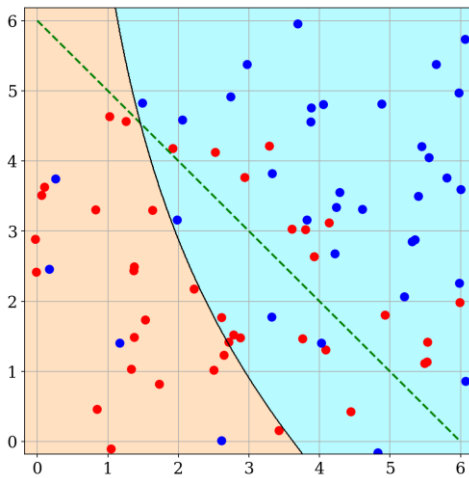
- The adjoints of last but one layer is calculated using (9):

$$\bar{f}_k^{L-1} = \sigma' \left(f_k^{L-1}(x_i) \right) w_k^L \bar{f}_p^L, \quad k = 1, 2, \dots, N_{L-1} \quad (10)$$

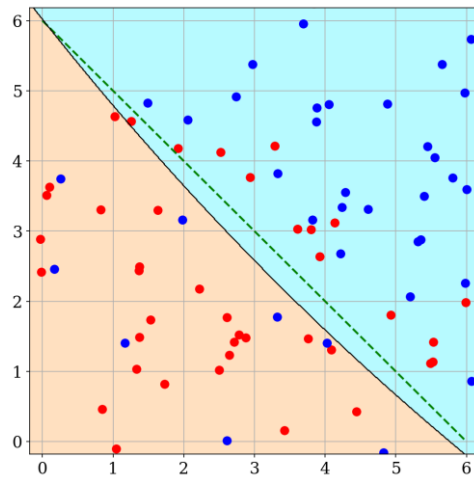
- The process is repeated to propagate errors back to all previous layers using (9).
- Once all the adjoints are found, the partial derivatives all the partial derivatives in (8).
- The process is repeated for all the training points in S_n .
- The weights are updated according to (7).

Neural Network (Example)

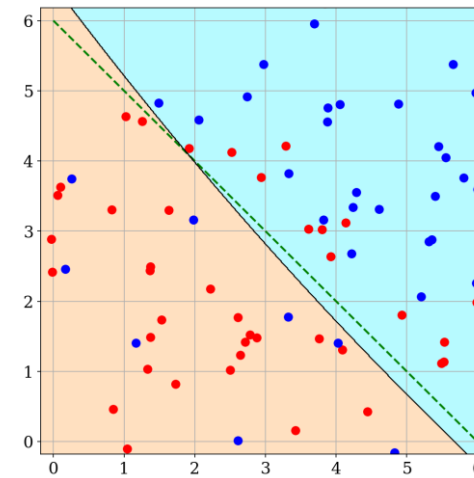
- Fully connected NN with one hidden layer and multiple neurons.
- One output neuron and logistic non-linearities.
- Log-loss function is used, maximum 1000 iterations.
- Underfitting with 15 neurons, close to optimum with 35, slight overfitting with more neurons.



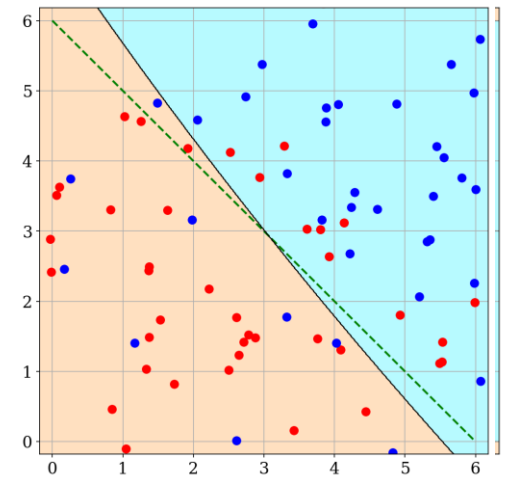
$H = 15$



$H = 35$



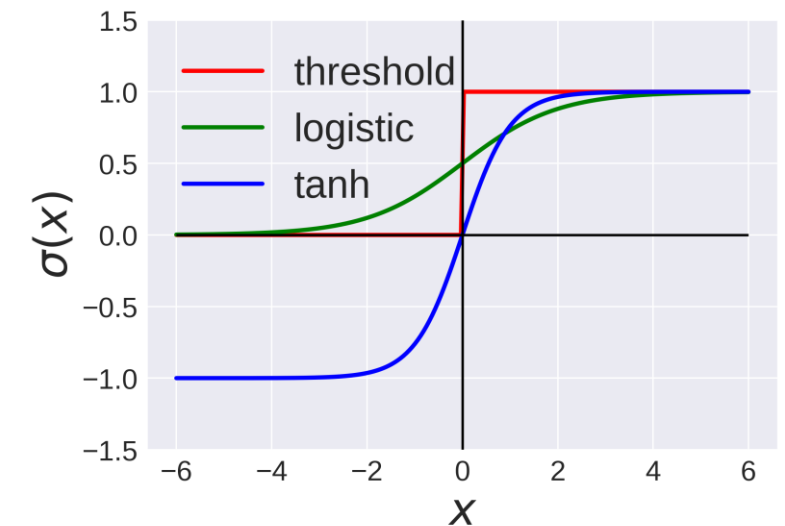
$H = 55$



$H = 75$

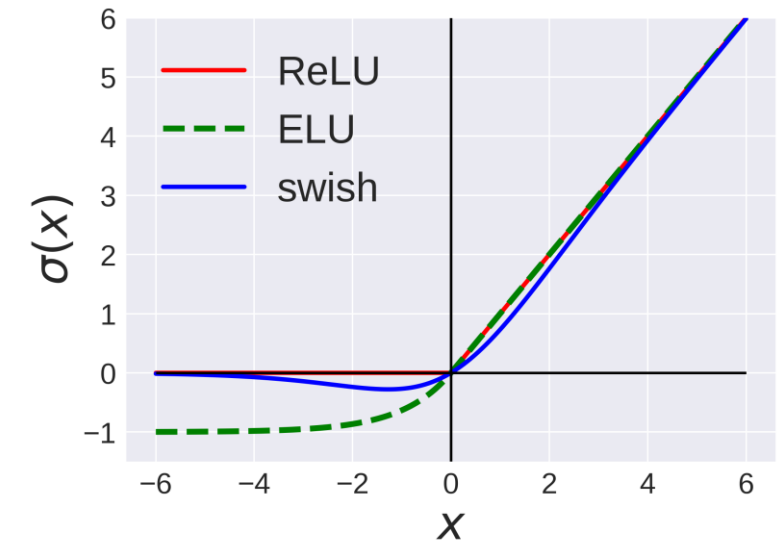
Learning in the deep: Saturating Non-linearities

- Standard backprop has a lot of problems.
- The derivative of the non-linearity affects the magnitude of pre-activation adjoints (as seen in [\(10\)](#)).
- If the derivative becomes small, the gradient in [\(8\)](#) becomes small, slowing down (even stalling) the training.
- This is known as the **vanishing gradient problem**.
- The problem worsens with larger number of layers resulting in initial layer weights never being really updated.
- Step linearity has no derivative at origin and 0 derivative everywhere else.
- Saturating non-linearities like tanh and logistic non-linearities produces very small derivatives at large magnitude pre-activations.



Learning in the deep: Non-saturating Non-linearities

- Non-saturating non-linearities deals with vanishing gradient problem.
- ReLU is the first non-saturating non-linearity proposed.
- However, ReLU is zero at non-zero values for negative activations causing **dead ReLU problem**.
- ReLU derivative is discontinuous at origin causing bouncing during training.
- ELU fixes this issue, but is only once differentiable at origin for $\alpha = 1$.
- Swish non-linearity has good properties from ReLU and ELU and is infinitely differentiable everywhere.



Learning in the deep: Weight Initialization

- Training starts with randomly chosen weight vector w_0 .
- Traditionally individual weights initialized as independent zero-mean($\mu = 0$) Gaussian random variables with unit variance ($\sigma^2 = 1$).
- This caused vanishing gradient problem because of difference in variance in the forward and backward direction when the adjacent layers had different number of neurons.
- **Xavier or Glorot** initialization is a common value for variance:

$$\sigma^2 = \frac{1}{n_{avg}} = \frac{2}{n_{in} + n_{out}}$$

- Empirically it was observed that this produced gradients with uniform variance over all layers in case of symmetric non-linearities.
- For non-symmetric non-linearities, **He initialization** is better, with variance:

$$\sigma^2 = \frac{2}{n_{in}}$$

Learning in the deep: Stochastic Gradient Descent

- When there is millions of data points, traditional backprop becomes infeasible.
- Running NNs in feedforward and backward mode for each training point is slow.
- **Stochastic gradient descent (mini-batch training)** takes a random sample of fixed number of training points and compute a noisy gradient.
- Increases training speed significantly.
- Empirical evidence suggests it helps gradient descent to escape poor local minima.

Learning in the deep: Learning Rate Schedule

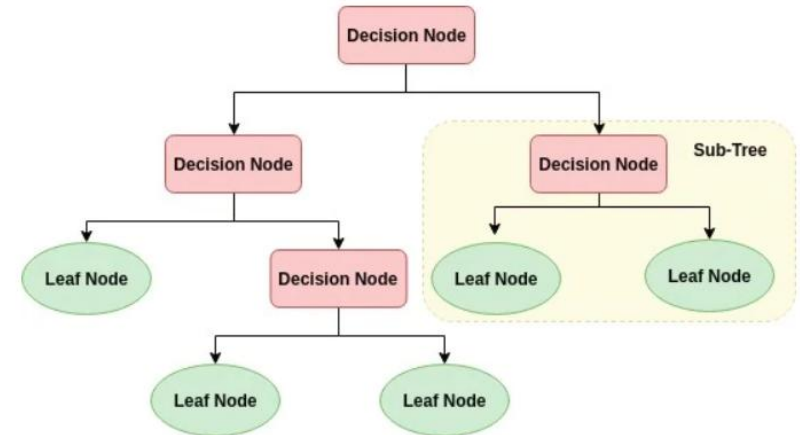
- The loss surface of deep NNs can be quite complex requiring careful selection of learning rates $\{\eta_t, t = 1, 2, \dots\}$
- The learning rate should typically be large in the beginning of training, to make quick progress.
- Learning rate should be smaller towards the end of training to allow convergence.
- If not, gradient descent will diverge (NaNs) or bounce around a sharp minima.
- Usually set before training starts.
- Example: exponential learning rate schedule:

$$\eta_t = \eta_0 0.1^{\frac{t}{s}}, \quad t = 0, 1, \dots$$

- Learning rate decreases by a factor of 10 every s steps.
- Another example: piecewise constant schedule (decide η_t for a fixed number of steps)

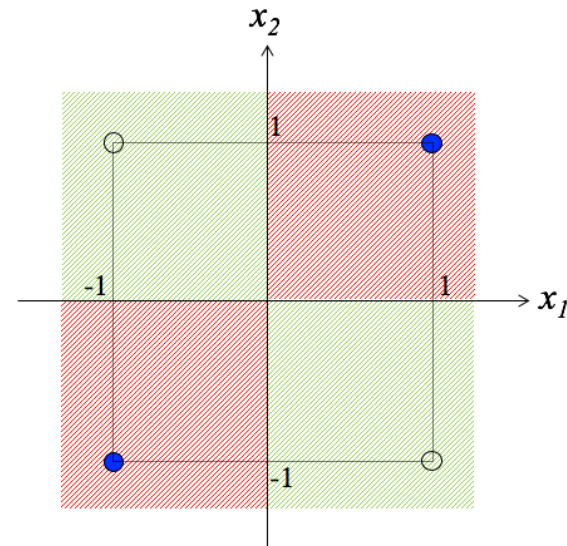
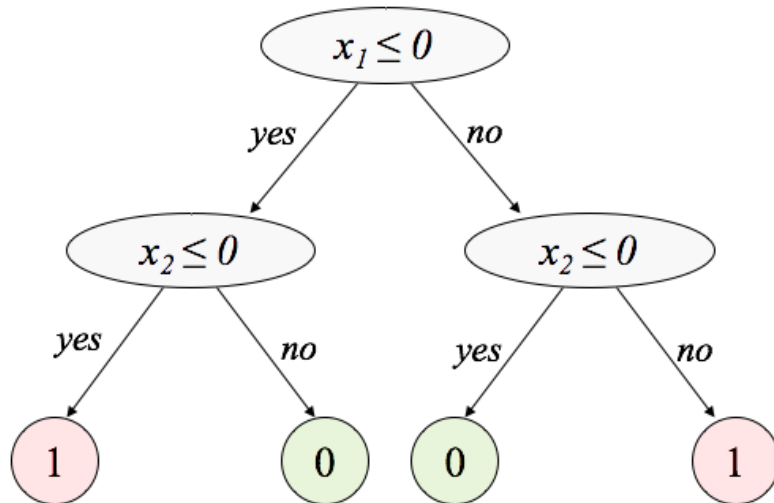
Decision Trees

- The main idea is to split the space recursively.
- Suitable for categorical as well as numerical features.
- Provides nonlinear, complex classifier with high degree of **interpretability**.
- A tree classifier consists of **splitting nodes** and **leaf nodes**.
- All tree classifiers have an initial splitting node known as **root node**.
- Every splitting node has at least two **descendants**.
- It consists of a hierarchy of nodes, where data splitting occurs.
- The terminal leaf node is where the assignments are made.
- The **depth** of a tree classifier is the maximum number of splits between the root node and any leaf node.
- In a **binary tree**, all splitting nodes have exactly two descendants.



Decision Trees – XOR Example

- Separating XOR data using a binary tree.
- Three splitting nodes and four leaf nodes.
- Classes can be described as:
Class 0 $\rightarrow ([x_1 \leq 0] \text{ AND } [x_2 > 0]) \text{ OR } ([x_1 > 0] \text{ AND } [x_2 \leq 0])$
Class 1 $\rightarrow ([x_1 \leq 0] \text{ AND } [x_2 \leq 0]) \text{ OR } ([x_1 > 0] \text{ AND } [x_2 > 0])$



Decision Trees

- To train a tree classifier from sample data, we consider **Classification and Regression Tree (CART)** rule.
- At each node, decision is made by evaluating:

$$x^j \leq \alpha$$

where x^j is one of the coordinates of x .

- This partitions the feature space into unions of rectangles.
- For each node, the coordinate to split on (j) and the split threshold (α) is trained using **impurity** criterion.
- Given a node R , let $N_i(R)$ be the number of training points from class i in R , $\forall i = 0, 1$.
- The total number of points in R :

$$N(R) = N_0(R) + N_1(R)$$

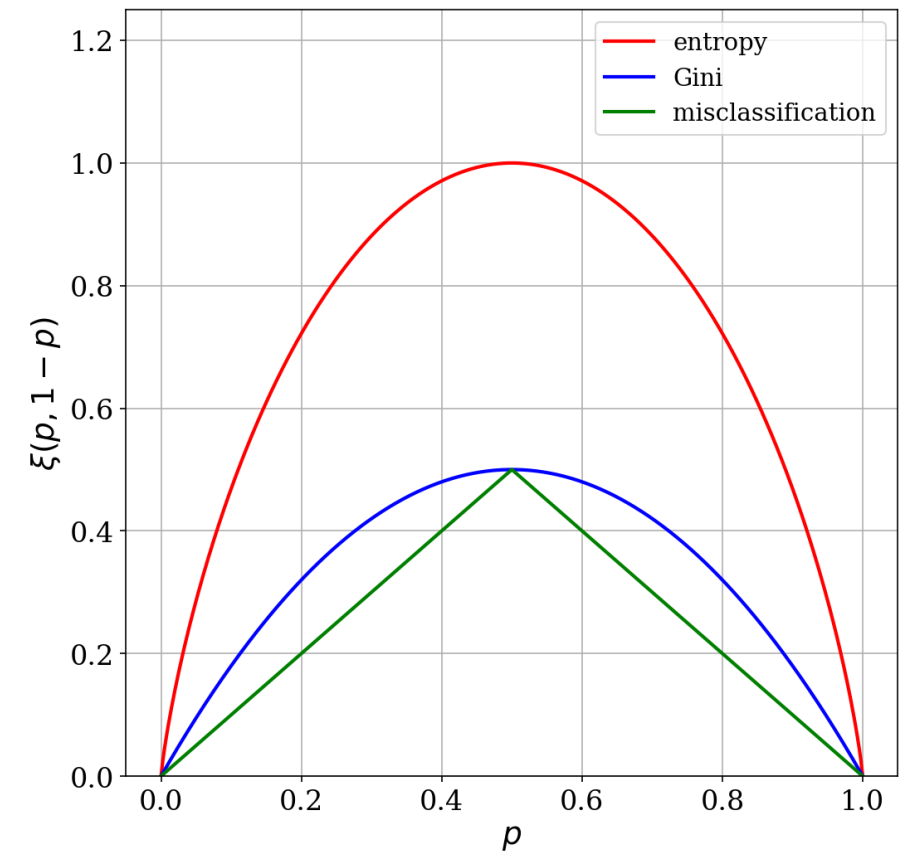
- The impurity of R is defined by:

$$\kappa(R) = \xi(p, 1 - p)$$

where $p = N_0(R)/N(R)$, $1 - p = N_1(R)/N(R)$.

Decision Trees

- The impurity function, $\xi(p, 1 - p)$ is non-negative and satisfies:
 - $\xi(0.5, 0.5) \geq \xi(p, 1 - p)$ for any $p \in [0, 1]$.
This means that, $\kappa(R)$ is maximum when $p = 1 - p = 0.5$ corresponding to maximum impurity.
 - $\xi(0, 1) = \xi(1, 0)$
 $\kappa(R) = 0$ if either $N_1(R) = 0$ or $N_0(R) = 0$. i.e. R is **pure**.
 - As a function of p , $\xi(p, 1 - p)$ increases for $p \in [0, 0.5]$ and decreases for $p \in [0.5, 1]$
 $\kappa(R)$ increases as the ratio $N_0(R)/N_1(R)$ approaches 1.
- Three common choices for the impurity function are:
 - The entropy impurity: $\xi(p, 1 - p) = -p \ln(p) - (1 - p) \ln(1 - p)$,
 - The Gini or variance impurity: $\xi(p, 1 - p) = 2p(1 - p)$,
 - The misclassification impurity: $\xi(p, 1 - p) = \min(p, 1 - p)$



Decision Trees

- For a node R , the coordinate j to split on and the split threshold α are determined as:

$R_{\alpha,-}^j$ = sub-rectangle resulting from split $x^j \leq \alpha$

$R_{\alpha,+}^j$ = sub-rectangle resulting from split $x^j > \alpha$

- The **impurity drop** is defined by:

$$\Delta_R(j, \alpha) = \kappa(R) - \frac{N(R_{\alpha,-}^j)}{N(R)} \kappa(R_{\alpha,-}^j) - \frac{N(R_{\alpha,+}^j)}{N(R)} \kappa(R_{\alpha,+}^j)$$

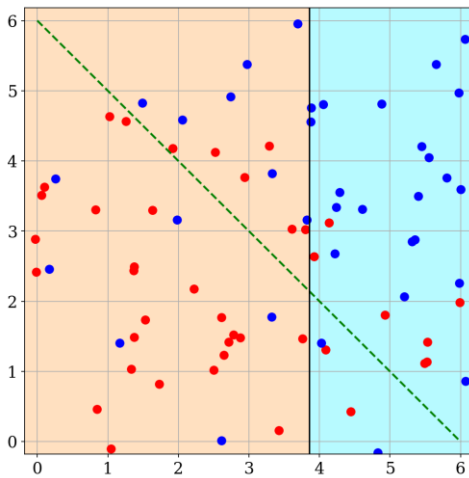
- The idea is to search for the j and α that would maximize the impurity drop.
- Since there are only finite number of data points and coordinates, there are only finite number of distinct candidate splits (j, α) .
- In a **fully-grown tree**, splitting stops when a **pure node** is found, which is declared a leaf node and given label of the points in it.
- Training stops when all current nodes are pure.

Decision Trees

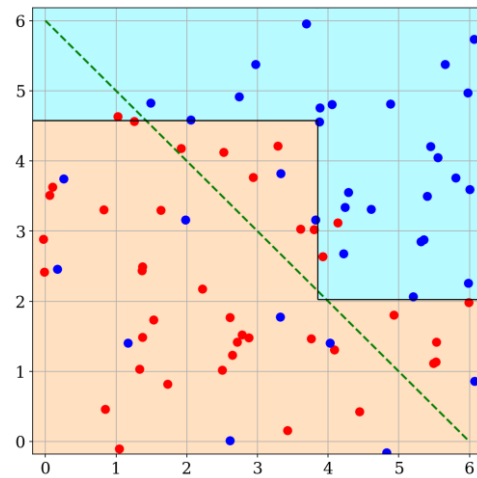
- A fully-grown tree results in overfitting.
- Regularization techniques should be employed to avoid overfitting:
 - **Stopped splitting:** Call a node leaf and assign majority label if:
 - There are fewer than specified number of points in the node,
 - The impurity drop is below a specified threshold,
 - A maximum tree depth have been reached, or
 - A maximum number of leaf nodes has been obtained.
 - **Pruning:** Fully grow the tree, then successively merge pairs of neighbouring leaves that increase impurity the least until a maximum impurity is reached.

Decision Trees (Example)

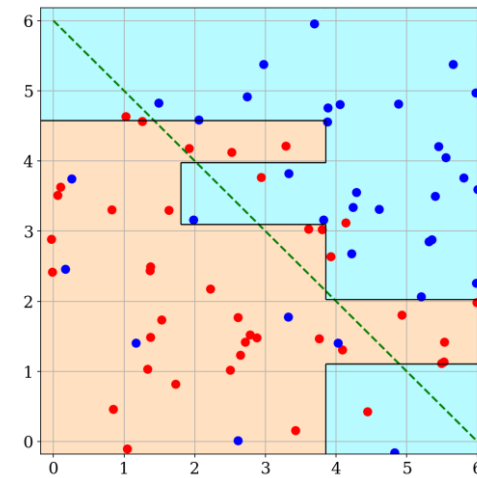
- CART classification rule with Gini impurity is employed.
- Regularization is employed, whereby no splits that leave fewer than s points in either child node are allowed.
- $s = 1$ corresponds to a fully-grown tree and shows overfitting
- $s = 20$: There is only one node (root node). The tree is called a **stump**.



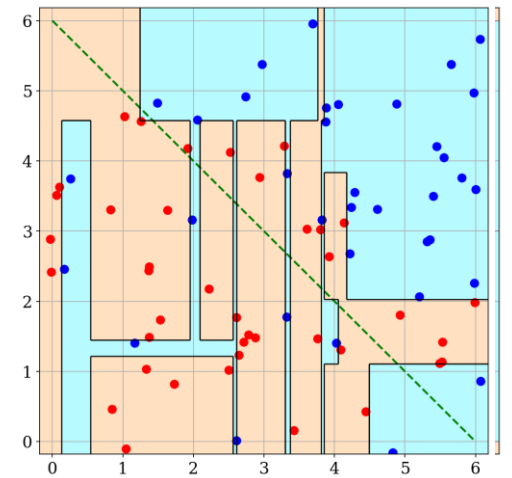
$s = 20$



$s = 8$



$s = 4$



$s = 1$

Thank You